

Verslag huistaak — User security

A) cron — wekelijkse controle van systeem users

Linux & Windows Server

Naam: Salaheddine Sennouni

R-nummer: R1050251

Klasgroep: B TI/2I4

Docent: Maarten Troost

Vak: Linux & Windows Server

Wat moet er gebeuren?

Elke maandag om 11u15 moet automatisch gecontroleerd worden of de lijst van systeem users (UID < 1000) veranderd is en of het paswoord van die users gewijzigd is. Bij verschillen moet ik mezelf inlichten, bij geen verschillen mag er niets gebeuren. De controle gebeurt via een cronjob en een bash-script. Voor de vergelijking gebruik ik diff (zoals in de hint van de opgave).

Voor de manier om mezelf in te lichten kies ik wall — dat broadcast een bericht naar alle ingelogde terminals zodat ik het zeker zie. Daarnaast schrijf ik elke gebeurtenis ook naar alerts.log zodat ik later kan terugkijken wat er wanneer gebeurd is.

Omgeving: Ubuntu 24.04 in een VMware-VM, gebruiker salaheddine-sennouni.

Stap 1 — Het script schrijven in nano

Ik open een nieuw bestand met nano in mijn home directory:

```
nano check_users.sh
```

Het script bestaat uit drie logische delen die ik hieronder één voor één bespreek.

Deel 1 — Lijst van systeem users en hun shadow-regel ophalen

Eerst filter ik met awk alle users met een UID < 1000 uit /etc/passwd. Dat zijn per definitie de systeem users (zie ook UID_MIN in /etc/login.defs). De namen schrijf ik weg in users.txt.

Daarna loop ik over die lijst en haal voor elke username de bijhorende regel uit /etc/shadow op met grep. Die regel bevat de username, de paswoord-hash en wat metadata. Alle gevonden regels samen vormen de huidige snapshot in nieuw.txt.

```
#!/bin/bash
# Wekelijkse controle van systeem users (UID < 1000)
# Salaheddine Sennouni - Linux & Windows Server

# 1) Maak lijst van alle systeem users en hun shadow regel
awk -F: '$3 < 1000 {print $1}' /etc/passwd > users.txt

> nieuw.txt
while read u; do
    grep "^$u:" /etc/shadow >> nieuw.txt
done < users.txt
```

Deel 1 van het script in nano: lijst opbouwen + shadow-regels ophalen.

Uitleg awk-syntax: -F: betekent splits elke regel op het ':' karakter (want /etc/passwd gebruikt ':' als scheidingsteken), '\$3 < 1000' is de voorwaarde (3e veld = UID is kleiner dan 1000), en {print \$1} drukt het eerste veld af (de username).

Deel 2 — Baseline aanmaken bij de eerste run

De eerste keer dat het script draait, bestaat oud.txt nog niet. Dan kopieer ik gewoon nieuw.txt naar oud.txt om een baseline te maken en stop ik met exit 0. Geen waarschuwing, want er is nog niets om mee te vergelijken. Ik schrijf ook één regel naar alerts.log zodat ik later weet wanneer de baseline opgezet is.

```
# 2) Eerste keer? Maak baseline en stop
if [ ! -f oud.txt ]; then
    cp nieuw.txt oud.txt
    echo "Baseline aangemaakt op $(date)" > alerts.log
    exit 0
fi
```

Deel 2 van het script: bij de eerste run wordt oud.txt aangemaakt als referentie.

Deel 3 — Vergelijken met diff en eventueel waarschuwen

Bestaat oud.txt wel? Dan vergelijk ik beide bestanden met diff. Door de output naar /dev/null te sturen kijk ik enkel naar de exit code: 0 betekent identiek, alles anders betekent dat er verschillen zijn.

Bij geen verschil eindigt het script met exit 0 — geen output, geen wall-bericht. Bij wel een verschil schrijf ik een blokje naar alerts.log met de tijdstip en de diff-output, en stuur ik dat blokje via wall naar alle ingelogde terminals. Tot slot update ik oud.txt zodat de nieuwe toestand de volgende referentie wordt.

```
# 3) Vergelijk oud met nieuw via diff
if diff oud.txt nieuw.txt > /dev/null; then
    # Geen verschil = geen waarschuwing
    exit 0
else
    # Wel verschil = waarschuw user
    echo "===== " >> alerts.log
    echo "WAARSCHUWING: systeem users gewijzigd!" >> alerts.log
    echo "Tijdstip: $(date)" >> alerts.log
    echo "Verschillen:" >> alerts.log
    diff oud.txt nieuw.txt >> alerts.log
    echo "===== " >> alerts.log

    wall < alerts.log

    # Update baseline
    cp nieuw.txt oud.txt
fi
```

Deel 3 van het script: vergelijking en waarschuwing via wall.

Volledige script en syntax-check

Na het opslaan met Ctrl+O en Ctrl+X check ik twee dingen: of het script correct opgeslagen is met cat, en of er geen syntax-fouten in zitten met bash -n. Die laatste optie laat bash de syntax controleren zonder het script effectief uit te voeren — handig om typo's te vangen voor je iets stoms doet als root.

```
cat check_users.sh
bash -n check_users.sh
```

```
salaheddine-sennouni@salaheddine-sennouni-VMware-Virtual-Platform:~$ nano check_users.sh
salaheddine-sennouni@salaheddine-sennouni-VMware-Virtual-Platform:~$ cat check_users.sh
#!/bin/bash
# Wekelijkse controle van systeem users (UID < 1000)
# Salaheddine Sennouni - Linux & Windows Server

# 1) Maak lijst van alle systeem users en hun shadow regel
awk -F: '($3 < 1000 {print $1})' /etc/passwd > users.txt

> nieuw.txt
while read u; do
    grep "^$u:" /etc/shadow >> nieuw.txt
done < users.txt

# 2) Eerste keer? Maak baseline en stop
if [ ! -f oud.txt ]; then
    cp nieuw.txt oud.txt
    echo "Baseline aangemaakt op $(date)" > alerts.log
    exit 0
fi

# 3) Vergelijk oud met nieuw via diff
if diff oud.txt nieuw.txt > /dev/null; then
    # Geen verschil = geen waarschuwing
    exit 0
else
    # Wel verschil = waarschuw user
    echo "===== " >> alerts.log
    echo "WAARSCHUWING: systeem users gewijzigd!" >> alerts.log
    echo "Tijdstip: $(date)" >> alerts.log
    echo "Verschillen:" >> alerts.log
    diff oud.txt nieuw.txt >> alerts.log
    echo "===== " >> alerts.log

    wall < alerts.log

    # Update baseline
    cp nieuw.txt oud.txt
fi

salaheddine-sennouni@salaheddine-sennouni-VMware-Virtual-Platform:~$ bash -n check_users.sh
salaheddine-sennouni@salaheddine-sennouni-VMware-Virtual-Platform:~$
```

Volledig script via cat + lege output van bash -n = syntax is OK.

Stap 2 — Script uitvoerbaar maken

Een nieuw bestand heeft standaard geen execute-permissie. Met `chmod +x` voeg ik die toe. Daarna controleer ik met `ls -l` of de x-jes erbij staan in de permissies.

```
chmod +x check_users.sh
ls -l check_users.sh
```

```
salaheddine-sennouni@salaheddine-sennouni-VMware-Virtual-Platform:~$ bash -n check_users.sh
salaheddine-sennouni@salaheddine-sennouni-VMware-Virtual-Platform:~$ chmod +x check_users.sh
salaheddine-sennouni@salaheddine-sennouni-VMware-Virtual-Platform:~$ ls -l check_users.sh
-rwxrwxr-x 1 salaheddine-sennouni salaheddine-sennouni 1051 May 14 16:51 check_users.sh
salaheddine-sennouni@salaheddine-sennouni-VMware-Virtual-Platform:~$
```

Rechten staan op -rwxr-xr-x — script is uitvoerbaar.

Stap 3 — Eerste run (baseline aanmaken)

Ik draai het script handmatig met sudo (anders geen leesrecht op /etc/shadow). Bij de eerste run moet de baseline aangemaakt worden zonder dat er een waarschuwing komt.

```
sudo ./check_users.sh
ls
cat alerts.log
wc -l users.txt
```

```
salaheddine-sennouni@salaheddine-sennouni-VMware-Virtual-Platform:~$ sudo ./check_users.sh
salaheddine-sennouni@salaheddine-sennouni-VMware-Virtual-Platform:~$ ls
.      check_users.sh  Documents  geenrechten  Music      oud.txt  Public  Templates  users.txt  werktst
alerts.log Desktop  Downloads  huistaakscripting.sh  nieuw.txt  Pictures  snap    testbestand  Videos
salaheddine-sennouni@salaheddine-sennouni-VMware-Virtual-Platform:~$ cat alerts.log
Baseline aangemaakt op Thu May 14 04:54:38 PM CEST 2026
salaheddine-sennouni@salaheddine-sennouni-VMware-Virtual-Platform:~$ wc -l users.txt
47 users.txt
salaheddine-sennouni@salaheddine-sennouni-VMware-Virtual-Platform:~$
```

Eerste run maakt users.txt, nieuw.txt, oud.txt en alerts.log. Het systeem heeft 47 systeem users.

De vier nieuwe bestanden staan er, alerts.log toont enkel "Baseline aangemaakt op..." en er zijn 47 systeem users gevonden. Geen waarschuwing — correct, want er was nog niks om mee te vergelijken.

Stap 4 — Test: geen wijziging = geen waarschuwing

Nu draai ik het script meteen opnieuw zonder iets te veranderen. Het mag geen output geven, geen wall-bericht, en niets nieuws toevoegen aan alerts.log. Dit bewijst de eis uit de opgave: "Als er geen verschillen zijn is alles ok".

```
sudo ./check_users.sh
echo "Exit code: $?"
cat alerts.log
```

```
salaheddine-sennouni@salaheddine-sennouni-VMware-Virtual-Platform:~$ sudo ./check_users.sh
salaheddine-sennouni@salaheddine-sennouni-VMware-Virtual-Platform:~$ echo "Exit code: $?"
Exit code: 0
salaheddine-sennouni@salaheddine-sennouni-VMware-Virtual-Platform:~$ cat alerts.log
Baseline aangemaakt op Thu May 14 04:54:38 PM CEST 2026
salaheddine-sennouni@salaheddine-sennouni-VMware-Virtual-Platform:~$
```

Script geeft geen output, exit code is 0, en alerts.log bevat nog steeds enkel de baseline-regel.

Resultaat klopt: geen output van het script zelf, exit code 0, en alerts.log is ongewijzigd.

Stap 5 — Test: nieuwe systeem user toevoegen

Nu test ik of het script een verandering wél oppikt. Met useradd -r voeg ik een nieuwe systeem user toe. De vlag -r (= --system) zorgt ervoor dat de gebruiker een UID krijgt onder UID_MIN (typisch tussen 100 en 999), dus binnen het bereik dat mijn script in de gaten houdt.

```
sudo useradd -r testsys
getent passwd testsys
```

```
sudo ./check_users.sh
cat alerts.log
```

```
salaheddine-sennouni@salaheddine-sennouni-VMware-Virtual-Platform:~$ sudo useradd -r testsys
salaheddine-sennouni@salaheddine-sennouni-VMware-Virtual-Platform:~$ getent passwd testsys
testsys:x:997:984::/home/testsys:/bin/sh
salaheddine-sennouni@salaheddine-sennouni-VMware-Virtual-Platform:~$ sudo ./check_users.sh

Broadcast message from root@salaheddine-sennouni-VMware-Virtual-Platform (pts/1):

Baseline aangemaakt op Thu May 14 04:54:38 PM CEST 2026
=====
WAARSCHUWING: systeem users gewijzigd!
Tijdstip: Thu May 14 04:56:57 PM CEST 2026
Verschillen:
47a48
> testsys:!:20587:~::~:
=====

salaheddine-sennouni@salaheddine-sennouni-VMware-Virtual-Platform:~$ cat alerts.log
Baseline aangemaakt op Thu May 14 04:54:38 PM CEST 2026
=====
WAARSCHUWING: systeem users gewijzigd!
Tijdstip: Thu May 14 04:56:57 PM CEST 2026
Verschillen:
47a48
> testsys:!:20587:~::~:
=====
salaheddine-sennouni@salaheddine-sennouni-VMware-Virtual-Platform:~$
```

Nieuwe user testsys krijgt UID 997. Broadcast verschijnt op het scherm en de diff-regel '> testsys:!:20587:~::~:' wordt toegevoegd aan alerts.log.

Een paar dingen om op te merken in deze output. Eerst: getent passwd testsys bevestigt dat de user UID 997 heeft, dus inderdaad een systeem user. Tweede: de diff toont 47a48 ("na regel 47 van oud is regel 48 toegevoegd in nieuw"). De toegevoegde regel is testsys:!:20587:~::~:. Het uitroepteken op de paswoord-positie betekent dat het account nog geen paswoord heeft (locked) — dat is normaal bij useradd -r want we hebben geen paswoord gezet.

Stap 6 — Test: paswoord van een systeem user wijzigen

Dit is de tweede eis uit de opgave: detecteren of het paswoord van een bestaande systeem user gewijzigd is. Ik geef testsys een paswoord via chpasswd (niet-interactieve manier om een paswoord te zetten, handig in scripts). De hash in /etc/shadow verandert dan van ! (locked) naar een echte \$y\$j9T\$... hash.

```
echo "testsys:MijnTestPaswd123" | sudo chpasswd
sudo grep testsys /etc/shadow | cut -c1-50
sudo ./check_users.sh
cat alerts.log
```

```

salaheddine-sennouni@salaheddine-sennouni-VMware-Virtual-Platform:~$ echo "testsys:MijnTestPaswd123" | sudo chpasswd
salaheddine-sennouni@salaheddine-sennouni-VMware-Virtual-Platform:~$ sudo grep testsys /etc/shadow | cut -c1-50
testsys:$y$j9T$YlPToiuyEi7D3tMaWLPoe/$7NDQLcPFFNwL
salaheddine-sennouni@salaheddine-sennouni-VMware-Virtual-Platform:~$ sudo ./check_users.sh

Broadcast message from root@salaheddine-sennouni-VMware-Virtual-Platform (pts/1)

Baseline aangemaakt op Thu May 14 04:54:38 PM CEST 2026
=====
WAARSCHUWING: systeem users gewijzigd!
Tijdstip: Thu May 14 04:56:57 PM CEST 2026
Verschillen:
47a48
> testsys:!:20587:~::~:
=====
WAARSCHUWING: systeem users gewijzigd!
Tijdstip: Thu May 14 04:57:56 PM CEST 2026
Verschillen:
48c48
< testsys:!:20587:~::~:
---
> testsys:$y$j9T$YlPToiuyEi7D3tMaWLPoe/$7NDQLcPFFNwLT8UZbzAP6Mt0e/q9Ls7VZJ1p8kl
idv2:20587:~::~:
=====

salaheddine-sennouni@salaheddine-sennouni-VMware-Virtual-Platform:~$ cat alerts.log
Baseline aangemaakt op Thu May 14 04:54:38 PM CEST 2026
=====
WAARSCHUWING: systeem users gewijzigd!
Tijdstip: Thu May 14 04:56:57 PM CEST 2026
Verschillen:
47a48
> testsys:!:20587:~::~:
=====
WAARSCHUWING: systeem users gewijzigd!
Tijdstip: Thu May 14 04:57:56 PM CEST 2026
Verschillen:
48c48
< testsys:!:20587:~::~:
---
> testsys:$y$j9T$YlPToiuyEi7D3tMaWLPoe/$7NDQLcPFFNwLT8UZbzAP6Mt0e/q9Ls7VZJ1p8klidv2:20587:~::~:
=====
salaheddine-sennouni@salaheddine-sennouni-VMware-Virtual-Platform:~$

```

Paswoord-wijziging gedetecteerd. Diff 48c48 toont dat regel 48 vervangen is — oude regel '<' met '!' en nieuwe regel '>' met de echte hash.

De diff is hier 48c48 ("regel 48 in oud is gewijzigd naar regel 48 in nieuw"). Dat is precies wat we willen zien voor een wijziging in plaats van een toevoeging: zelfde regelpositie, andere inhoud. Door `cut -c1-50` toon ik enkel het begin van de hash; de volledige hash is veel langer. Het `alerts.log` bevat nu drie gebeurtenissen op chronologische volgorde: de baseline-aanmaak, de toegevoegde user, en de paswoord-wijziging.

Stap 7 — Cronjob installeren (elke maandag 11u15)

Voor de cronjob open ik de crontab van root met `sudo crontab -e`. Root, omdat het script `/etc/shadow` moet kunnen lezen. Nano start op (eerste keer kreeg ik een vraag welke editor ik wou, daar koos ik 1 = nano) en ik voeg onderaan één regel toe:

```
15 11 * * 1 /home/salaheddine-sennouni/check_users.sh
```

Uitleg van de cron-expressie: 15 = minuut, 11 = uur, * = elke dag van de maand, * = elke maand, 1 = weekday 1 (zondag is 0 in cron, dus 1 is maandag). Samen: elke maandag om 11u15.

```
salaheddine-sennouni@salaheddine-sennouni-VMware-Virtual-Platform: ~
GNU nano 7.2 /tmp/crontab.SyuhLa/crontab *
# Edit this file to introduce tasks to be run by cron.
#
# Each task to run has to be defined through a single line
# indicating with different fields when the task will be run
# and what command to run for the task
#
# To define the time you can provide concrete values for
# minute (m), hour (h), day of month (dom), month (mon),
# and day of week (dow) or use '*' in these fields (for 'any').
#
# Notice that tasks will be started based on the cron's system
# daemon's notion of time and timezones.
#
# Output of the crontab jobs (including errors) is sent through
# email to the user the crontab file belongs to (unless redirected).
#
# For example, you can run a backup of all your user accounts
# at 5 a.m every week with:
# 0 5 * * 1 tar -zcf /var/backups/home.tgz /home/
#
# For more information see the manual pages of crontab(5) and cron(8)
#
# m h dom mon dow  command
15 11 * * 1 /home/salaheddine-sennouni/check_users.sh
```

Cron-regel toegevoegd in nano. De commentaar bovenaan komt standaard mee bij `sudo crontab -e`.

Bevestigen dat de cronjob effectief geïnstalleerd is

Met `sudo crontab -l` lees ik de crontab van root uit en check ik of mijn regel erin staat. Met `systemctl is-active cron` controleer ik of de cron-daemon ook effectief draait — anders wordt mijn job niet uitgevoerd.

```
sudo crontab -l
systemctl is-active cron
```

```
salaheddine-sennouni@salaheddine-sennouni-VMware-Virtual-Platform:~$ sudo crontab -l
# Edit this file to introduce tasks to be run by cron.
#
# Each task to run has to be defined through a single line
# indicating with different fields when the task will be run
# and what command to run for the task
#
# To define the time you can provide concrete values for
# minute (m), hour (h), day of month (dom), month (mon),
# and day of week (dow) or use '*' in these fields (for 'any').
#
# Notice that tasks will be started based on the cron's system
# daemon's notion of time and timezones.
#
# Output of the crontab jobs (including errors) is sent through
# email to the user the crontab file belongs to (unless redirected).
#
# For example, you can run a backup of all your user accounts
# at 5 a.m every week with:
# 0 5 * * 1 tar -zcf /var/backups/home.tgz /home/
#
# For more information see the manual pages of crontab(5) and cron(8)
#
# m h dom mon dow  command
15 11 * * 1 /home/salaheddine-sennouni/check_users.sh
salaheddine-sennouni@salaheddine-sennouni-VMware-Virtual-Platform:~$ 15 11 * * 1 /home/salaheddine-sennouni/check_users.sh
15: command not found
salaheddine-sennouni@salaheddine-sennouni-VMware-Virtual-Platform:~$ systemctl is-active cron
active
salaheddine-sennouni@salaheddine-sennouni-VMware-Virtual-Platform:~$
```

Cron-regel staat in de crontab van root, cron-service is active.

Klein zijstapje: in de screenshot zie je halverwege een 'command not found' fout. Dat was omdat ik per ongeluk de cron-regel zelf in de terminal getypt heb in plaats van enkel in nano. Heeft geen invloed op de cronjob zelf — die is correct geïnstalleerd zoals crontab -l aantoont.

Conclusie

Het script doet wat in de opgave gevraagd werd. De lijst van systeem users wordt bepaald via `UID < 1000`, de wekelijkse controle gebeurt via een cronjob in root's crontab op maandag 11u15, en de vergelijking gebeurt met diff zoals de hint in de opgave aangaf. Bij wijzigingen krijg ik via wall een broadcast op mijn scherm, en de wijziging wordt ook gelogd in alerts.log. Bij geen wijzigingen blijft het script stil — geen output, geen waarschuwing, niks toegevoegd aan de log.

De vier scenario's bewijzen samen dat het script werkt: de baseline wordt correct aangemaakt op de eerste run (Stap 3), een tweede run zonder wijzigingen geeft geen waarschuwing (Stap 4), een toegevoegde systeem user wordt opgepikt (Stap 5), en een gewijzigd paswoord wordt eveneens gedetecteerd (Stap 6). De diff-output toont in beide laatste gevallen exact wat er veranderd is, wat handig is voor mij om snel te zien of het iets normaal is of iets verdachts.